สรุป INT161 Basic Backend Development

ตั้งแต่ HTTP ไปจนถึง REST API แบบแบ่งเลเยอร์ด้วย Express + MySQL พร้อม Error Handling, Pagination และ DTO เรียบเรียงจากสไลด์และคาบเรียนกลุ่ม B (วันพฤหัส) มีตัวทดลองให้กดเล่นและแบบฝึกหัดท้ายบท

Node.js · HTTP · REST · Express · mysql2 · Error Handling · Pagination · DTO

```
# HTTP Request
GET /api/products?page=2&limit=20
HTTP/1.1
Host: localhost:3000
Accept: application/json

# HTTP Response
HTTP/1.1 200 OK
Content-Type: application/json

{ "success": true, "data": [ … ],
  "pagination": { "page": 2,
"totalPages": 7 } }
```

สอบครั้งที่ 2

19–26 ต.ค. 2569

ทฤษฎี 30% + ปฏิบัติ 20% · สอบรวม 3 ชั่วโมง

ทฤษฎี · 1 ชั่วโมง

ปรนัย 5 ตัวเลือก

ตอบถูกข้อเดียว ฝนกระดาษคำตอบ (เตรียมดินสอ 2B) ออกทุกเรื่องที่เรียนมา

ปฏิบัติ · 2 ชั่วโมง

เขียน API ตาม spec

ใช้ template ของแต่ละคน ตรวจด้วย test runner · ปิดเน็ตหลัง 15 นาทีแรก ดู เช็กลิสต์

ภาพรวมทั้งวิชา

Request หนึ่งครั้ง วิ่งผ่านอะไรบ้าง

ทุกบทในวิชานี้คือชิ้นส่วนหนึ่งของเส้นทางนี้ ด้านล่างคือแต่ละขั้นที่ request ผ่าน ทั้งกรณีปกติและกรณีเกิด error

เส้นทางปกติ (POST สร้าง brand สำเร็จ)

ขั้นที่ 1 / 9 · เส้นทางปกติ

Client ส่ง request

Front-end หรือ test runner ยิง request มาพร้อม method, path, query string และ body (JSON)

```
POST /api/66130500001/brands?page=1&limit=5
Content-Type: application/json

{ "name": "Pulsar", "country": "Korea" }
```

ขั้นที่ 2 / 9 · เส้นทางปกติ

express.json() แปลง body

built-in middleware อ่าน stream ของ body แล้ว JSON.parse ให้ ผลลัพธ์อยู่ใน req.body จากนั้นเรียก next()

```
app.use(express.json())
// req.body = { name: "Pulsar", country: "Korea" }
```

ขั้นที่ 3 / 9 · เส้นทางปกติ

queryParser() คำนวณ pagination

middleware ของเราเองอ่าน page/limit จาก req.query ตรวจค่า แล้วแนบ req.pagination ไว้ให้ชั้นถัดไป

```
req.pagination = { page: 1, limit: 5, offset: 0 }
```

ขั้นที่ 4 / 9 · เส้นทางปกติ

Router จับคู่ path + method

Controller ดึงค่าจาก req.params / req.query / req.body ตรวจ input เบื้องต้น แล้วส่งต่อให้ service ไม่มี SQL หรือ business logic ในชั้นนี้

```
router.post('/', async (req, res, next) ⇒ {
  const brand = await service.create(req.body)
  apiResponse(res, brand, null, 201)
})
```

ขั้นที่ 5 / 9 · เส้นทางปกติ

Service ทำ business logic

ตรวจกฎทางธุรกิจ เช่น ชื่อห้ามว่าง ประสานหลาย repository ถ้าต้องใช้ แล้วแปลงผลด้วย mapper ไม่รู้จัก req/res

```
if (!data.name) throw new AppError('name is required', 400)
const entity = await repo.create(data)
return mapper.map(entity)
```

ขั้นที่ 6 / 9 · เส้นทางปกติ

Repository คุยกับฐานข้อมูล

ชั้นเดียวที่เขียน SQL ใช้ prepared statement (?) เสมอ

```
const [result] = await pool.query(
  'INSERT INTO brands SET ?', [data])
return findById(result.insertId)
```

ขั้นที่ 7 / 9 · เส้นทางปกติ

MySQL ประมวลผล

DB ตอบกลับเป็น ResultSetHeader (insertId, affectedRows) หรือ array ของแถว

```
{ affectedRows: 1, insertId: 21 }
```

ขั้นที่ 8 / 9 · เส้นทางปกติ

Mapper แปลง Entity → DTO

เลือกเฉพาะ field ที่ควรส่งออก เปลี่ยนชื่อ field ตามสัญญาของ API ตัดข้อมูลลับทิ้ง

```
{ id: 21, name: "Pulsar", country: "Korea" }
```

ขั้นที่ 9 / 9 · เส้นทางปกติ

ส่ง response กลับ

apiResponse ห่อ data ด้วยรูปแบบมาตรฐาน พร้อม status ที่ถูกต้อง (POST สำเร็จ = 201)

```
HTTP/1.1 201 Created
{ "success": true, "data": { ... }, "timestamp": "2026-10-21T11:00:00.000Z" }
```

เส้นทาง error (GET id ที่ไม่มี → 404)

ขั้นที่ 1 / 6 · เส้นทาง error

Client ขอสิ่งที่ไม่มี

ขอ brand id ที่ไม่มีในฐานข้อมูล

```
GET /api/66130500001/brands/999
```

ขั้นที่ 2 / 6 · เส้นทาง error

Router ส่งต่อ

Controller เรียก service ตามปกติ โดยห่อด้วย try/catch (หรือพึ่ง Express 5 ที่ส่ง async error ต่อให้เอง)

```
try { apiResponse(res, await service.getById(id)) }
catch (e) { next(e) }
```

ขั้นที่ 3 / 6 · เส้นทาง error

Service ไม่ catch

ปล่อยให้ error ไหลขึ้นไปตาม call stack (error propagation)

```
return mapper.map(await repo.findById(id))
```

ขั้นที่ 4 / 6 · เส้นทาง error

Repository โยน AppError

query ได้ 0 แถว จึง throw error ที่ระบุ status ไว้ในตัว

```
if (rows.length === 0)
  throw new AppError(`Brand 999 not found`, 404)
```

ขั้นที่ 5 / 6 · เส้นทาง error

next(err) กระโดดไป error middleware

Express ข้าม middleware/route ปกติทั้งหมด ไปหาฟังก์ชันที่มี 4 parameter ที่วางไว้ท้ายสุด

```
app.use(errorHandler)  // (err, req, res, next)
```

ขั้นที่ 6 / 6 · เส้นทาง error

ตอบ error ที่อ่านเข้าใจ

ส่ง status code ที่ถูกต้องและ JSON ที่ไม่มี stack trace หลุดออกไป

```
HTTP/1.1 404 Not Found
{ "error": "NOT_FOUND", "statusCode": 404,
  "message": "Brand 999 not found",
  "path": "/api/66130500001/brands/999" }
```

สัปดาห์ 1 · 6 ส.ค.

Web Application, JSON และ Node.js

รู้ว่า backend อยู่ตรงไหนของเว็บ คุยกับ client ด้วยอะไร และทำไมใช้ Node.js

สถาปัตยกรรมเว็บ 2 แบบ

MPA (Multi-Page Application)

เว็บแบบดั้งเดิม ทุกครั้งที่เปลี่ยนหน้า เบราว์เซอร์โหลดหน้าใหม่จาก server ทั้งหน้า (full page refresh) server ใช้ **template engine** สร้าง HTML ส่งกลับมา

Router → Controller → Model → View (Template) → HTML

SPA (Single Page Application)

โหลด HTML+CSS+JS ครั้งแรกครั้งเดียว หลังจากนั้นหน้าเว็บ (หรือแอปมือถือ) ขอข้อมูลจาก **RESTful API** ด้วย AJAX / fetch แล้วได้ **JSON** กลับมา วิชานี้เราเขียนฝั่งนี้

Front-end ⇄ {JSON} ⇄ Back-end (REST API) ⇄ DB

JSON (JavaScript Object Notation)

- รูปแบบแลกเปลี่ยนข้อมูลที่เบา คนอ่านง่าย เครื่อง parse ง่าย
- มาจาก syntax ของ JavaScript (ECMA-262 3rd Edition) แต่**ไม่ผูกกับภาษาใด** ใช้ได้ทุกภาษา
- ใน Node: `JSON.stringify(obj)` แปลง object → ข้อความ, `JSON.parse(text)` แปลงข้อความ → object

Web Services: SOAP กับ REST

	SOAP	RESTful
คืออะไร	Simple Object Access Protocol เป็น **โปรโตคอล**	เป็น**แนวทางการออกแบบ (design approach)** ไม่ใช่โปรโตคอล
รูปแบบข้อมูล	XML มาตรฐาน อธิบายด้วย WSDL	ใช้ได้หลายแบบ (text, XML) นิยม JSON ที่สุด
ผู้คิด	—	Roy Fielding (คนเดียวกับที่ร่วมออกแบบ HTTP)

ตัวแปลภาษา 3 แบบ

Compiler

แปลทั้งโปรแกรมเป็น machine code ล่วงหน้า

- รันเร็วที่สุด
- ไฟล์ผูกกับ OS/CPU (portable น้อย)
- ไม่ต้องแจก source code

C, C++, Go

Interpreter

อ่านและแปลทีละบรรทัดตอนรัน

- พัฒนา/debug เร็ว ไม่ต้องรอ compile
- ช้ากว่า (แปลไปรันไป)
- portable สูง แต่ต้องแจก source

JavaScript, Python

Hybrid

compile เป็น **byte code** ก่อน แล้วให้ VM (interpreter + JIT) แปลเป็น machine code ตอนรัน

Java, Kotlin, C# (.NET)

Node.js คืออะไร

> **Node.js ไม่ใช่ภาษา** แต่เป็น **runtime** ที่ทำให้รัน JavaScript ฝั่ง server ได้ เป็น open source ฟรี รันได้หลาย OS ข้อดีคือใช้ภาษาเดียว (JS) ทั้ง front-end และ back-end และทำงานแบบ asynchronous บน single thread (async/await)

my-third-app.js · HTTP server แบบดิบ

```
const http = require('node:http');

const listener = function (request, response) {
  if (request.url === '/user') {
    response.writeHead(200, { 'Content-Type': 'application/json' });
    response.end(JSON.stringify({ name: 'John Doe' }));
  } else if (request.url === '/') {
    response.writeHead(200);
    response.end('Hello World');
  } else {
    response.writeHead(404);
    response.end('Error: Page Not Found');
  }
};

const server = http.createServer(listener);
server.listen(3000);
console.log('Server running at http://127.0.0.1:3000/');
```

สามส่วนหลักของ Node web app: (1) import module ด้วย require/import (2) สร้าง server ที่ listen รอ client (3) อ่าน request แล้วส่ง response กลับ

สัปดาห์ 2 · 13 ส.ค.

HTTP Programming Basics

โครงสร้าง request/response, โมดูล http ของ Node, CJS กับ ESM และการอ่านค่าจาก URL

HTTP Request เก็บข้อมูลได้ 3 ที่

- **Method** บอกว่าจะทำอะไร (GET, POST, ...)
- **Headers** ข้อมูลประกอบ เช่น `Content-Type`, `Host`
- **Body** ข้อมูลหลัก เช่น JSON ตอน POST/PUT

ส่วน URL เองก็มี path และ query string ด้วย

HTTP Response มี 3 ส่วน

- **Status line** บรรทัดแรก เช่น `HTTP/1.1 200 OK`
- **Headers**
- **Message body** (payload)

Node.js ทำงานยังไง (Event Loop)

1. client ส่ง request เข้ามา (อาจเป็นงาน blocking หรือ non-blocking เช่น query/update/delete)
2. Node รับ request ไปต่อคิวใน **Event Queue**
3. **Event Loop** หยิบทีละอัน ถ้าเป็นงานง่ายที่ไม่ต้องรอทรัพยากรภายนอก (non-blocking) ก็ทำแล้วตอบกลับเลย

Core modules ที่ควรรู้จัก

โมดูล	ใช้ทำอะไร	โมดูล	ใช้ทำอะไร
`http`	สร้าง HTTP server/client	`fs`	อ่าน/เขียนไฟล์
`path`	จัดการ path ไฟล์	`os`	ข้อมูลระบบปฏิบัติการ
`events`	จัดการ event	`stream`	จัดการข้อมูลแบบ stream
`crypto`	เข้ารหัส/แฮช	`url`, `querystring`	แยกส่วน URL และ query string

CommonJS กับ ES Modules

หัวข้อ	CommonJS (CJS)	ES Modules (ESM) ✓ แนะนำ
Syntax	`require()` / `module.exports`	`import` / `export`
การโหลด	synchronous ตอน runtime	parse แบบ static รองรับ async
ใช้ที่ไหน	Node.js (ระบบเก่า)	ทั้งเบราว์เซอร์และ Node.js
Tree-shaking	ทำได้ไม่ดี	bundler ตัดโค้ดที่ไม่ใช้ออกได้
ตำแหน่งไฟล์	`__dirname`, `__filename`	`import.meta.url`

> **อย่าลืม** จะใช้ `import` ใน Node ต้องใส่ `"type": "module"` ใน package.json (หรือใช้ไฟล์ .mjs)

4 คลาสหลักของ node:http

`http.Server`
`server.listen(port, cb)` เริ่มรอ connection · `server.close()` · event `'request'` เกิดทุกครั้งที่มี request

`http.ServerResponse` (res)
`writeHead(status, headers)` · `setHeader(name, value)` · `write(chunk)` · `end(data)` บอกว่าส่งครบแล้ว

`http.IncomingMessage` (req)
`req.method` · `req.url` · `req.headers` เป็น ReadableStream (body ต้องอ่านเป็น chunk)

`http.ClientRequest`
เกิดตอนเรียก `http.request()` ใช้ยิง request ไปหา server อื่น · event `'response'`

อ่านค่าจาก URL

http-server.js

```
import * as http from 'node:http';

const listener = (request, response) => {
  const urlObj = new URL(request.url, `http://${request.headers.host}`);

  // query string: ?name=Somchai&fav_subj=A&fav_subj=B
  const name = urlObj.searchParams.get('name');          // 'Somchai'
  const subjects = urlObj.searchParams.getAll('fav_subj'); // ['A', 'B']

  // path variable: /users/123
  const segments = urlObj.pathname.split('/').filter(Boolean); // ['users', '123']
```

```
  response.writeHead(200, { 'Content-Type': 'text/plain' });
  response.end(`You made a ${request.method} request to ${request.url}`);
};

http.createServer(listener).listen(3000);
```

ทดลองแยก URL

INTERACTIVE

พิมพ์ URL

```
http://localhost:3000/api/subjects/5?page=2&limit=10&fav_subj=Backend&fav_subj=
```

```
pathname            → /api/subjects/5
split('/')          → ["","api","subjects","5"]
.filter(Boolean)    → ["api","subjects","5"]

searchParams.get('page') → "2"
searchParams.get('limit') → "10"
searchParams.getAll('fav_subj') → ["Backend","Frontend"]

// ใน Express: app.get('/api/subjects/:id')
req.params → {"id":"5"}
req.query  → {"page":"2","limit":"10","fav_subj":["Backend","Frontend"]}
```

ทำไม `filter(Boolean)` ? `'/users/123'.split('/')` ได้ `['', 'users', '123']` ตัวแรกเป็นสตริงว่าง `Boolean('')` เป็น false จึงถูกกรองทิ้ง เหลือ `['users', '123']`

สัปดาห์ 3 · 20 ส.ค.

Introduction to RESTful API

REST = **RE**presentational **S**tate **T**ransfer โดย Roy Fielding · server เปิดให้เข้าถึง **resource** แต่ละ resource ระบุด้วย URI และส่งเป็น representation (นิยม JSON)

6 หลักการ (constraints) ของ REST

1. Client–Server

แยก client กับ server ออกจากกัน server มี web service ให้บริการ client ส่ง request มา server จะปฏิเสธหรือตอบสนองก็ได้

2. Stateless

ทุก request ต้อง**มีข้อมูลครบในตัวเอง** server ห้ามอาศัย context ที่เก็บไว้จาก request ก่อนหน้า **client ต้องเก็บ session state เอง**

3. Uniform Interface

ใช้ HTTP verb มาตรฐานจัดการ resource: POST สร้าง, GET อ่าน, PUT แก้ไข, DELETE ลบ (รายละเอียดด้านล่าง)

4. Layered System

ระบบแบ่งเป็นชั้น แต่ละชั้น**เห็นแค่ชั้นที่ติดกัน** client ไม่รู้ว่าคุยกับ server ปลายทางหรือตัวกลาง

5. Cacheable

response ต้องบอก (ทางตรงหรืออ้อม) ว่า cache ได้หรือไม่ ถ้าได้ client นำ response มาใช้ซ้ำกับ request เดียวกันในช่วงเวลาที่กำหนด

6. Code on Demand (optional)

server ส่งโค้ด (script/applet) ให้ client รันเพื่อขยายความสามารถได้ **เป็นข้อเดียวที่ไม่บังคับ**

Uniform Interface: 4 เสาหลัก

เสาหลัก	ความหมาย
Resource Identification	URI บอก **resource (คำนาม)** ไม่ใช่ action เช่น `/api/v1/users`
Manipulation via Representations	แลกเปลี่ยน state ด้วยรูปแบบมาตรฐาน (JSON)
Self-Descriptive Messages	ใช้ HTTP method และ header มาตรฐาน เช่น `Content-Type: application/json`
HATEOAS	ใส่ลิงก์ (hypermedia) ใน response เพื่อบอก client ว่าทำอะไรต่อได้

ออกแบบ endpoint ให้ถูกหลัก

Method	URI	ความหมาย
GET	`/api/products`	ดึงสินค้าทั้งหมด (collection)
GET	`/api/products/1`	ดึงสินค้า id = 1 (singleton)
GET	`/api/products/1/orders`	ดึง orders ของสินค้า id = 1 (sub-collection)
POST	`/api/products`	เพิ่มสินค้าใหม่ (ส่ง body)
PUT	`/api/products/1`	แก้ไขสินค้า id = 1 (ส่ง body)
DELETE	`/api/products/1`	ลบสินค้า id = 1

✓ ควรทำ

- URI ลงท้ายด้วย**คำนาม** และใช้**พหูพจน์** เสมอ (`/users`)
- ให้ HTTP verb เป็นตัวบอก action
- ส่ง status code ที่ถูกต้องทั้งตอนสำเร็จและ error
- ดู endpoint + method แล้วรู้ทันทีว่าทำอะไร

× ไม่ควรทำ

- `/getUsers` , `/createProduct` (มีคำกริยาใน URI)
- `/user/1` ปนกับ `/users` (เอกพจน์ปนพหูพจน์)
- ตอบ 200 ทุกกรณีแล้วใส่ error ไว้ใน body

Layered System ในโค้ด

```
src/
├── server.js        → ตั้งค่า HTTP server
├── router.js        → จับคู่ route + จัดการ HTTP (Controller)
├── services/        → business logic (ไม่ผูกกับ framework)
└── repositories/    → เข้าถึงข้อมูล (ไฟล์ / DB)
```

ประโยชน์ของการแบ่งชั้น: Scalability (เพิ่ม cache/load balancer ได้โดย client ไม่ต้องแก้), Separation of Concerns (แยกการ parse HTTP ออกจาก business logic และ database) และ Security

สัปดาห์ 4 · 27 ส.ค.

Express.js Framework

web framework ที่ minimalist, unopinionated และเร็ว ครอบ http ของ Node ไว้ด้วยระบบ routing และ middleware เป็นตัว E ใน MEAN/MERN stack

งาน	Node http ดิบ	Express
อ่าน body	ต่อ chunk เองจาก stream (`req.on('data')`)	`app.use(express.json())` แล้วใช้ `req.body`
Routing	if/else เช็ก `req.url` เอง	`app.get('/api/subjects/:id', …)`
อ่าน URL/query	`new URL()` + `searchParams`	`req.params`, `req.query`
ส่ง response	`res.writeHead()` + `res.end()`	`res.status(201).json(obj)`

3 ที่มาของข้อมูลใน request

```
PUT /api/subjects/5?filter=INT&page=1&size=10
```

```
import express from 'express';
const app = express();
app.use(express.json());                // middleware: parse JSON body → req.body

app.put('/api/subjects/:id', (req, res) => {
  const id = req.params.id;                    // '5'      ← จาก path (:id)
  const { filter, page, size } = req.query;    // 'INT','1','10' ← จาก ?query
  const subject = req.body;                    // { code, title, credit } ← จาก body
  subject.id = id;
  res.status(200).json(subject);
});

app.listen(3000);
```

> **กับดัก:** ค่าใน `req.params` และ `req.query` เป็น **string เสมอ** ถ้าจะคำนวณต้อง `parseInt()` / `Number()` ก่อน และ `res.status()` อย่างเดียว**ยังไม่ส่ง response** ต้องต่อด้วย `.json()` หรือ `.send()`

Middleware

Middleware คือฟังก์ชัน `(req, res, next)` ที่ทำงาน**ก่อนถึง router** เช่น log, parse body, ตรวจสิทธิ์, validate, เปิด CORS ทำงานเรียงตามลำดับที่ `app.use()`

- เรียก `next()` → ส่งต่อให้ middleware ตัวถัดไป
- เรียก `res.send()` / `res.json()` → จบ ส่ง response กลับ
- เรียก `next(err)` → ข้ามไป error-handling middleware ทันที

ประเภท: Application-level, Router-level, Error-handling, Built-in (`express.json()`), Third-party

หน้าที่ของแต่ละเลเยอร์

เลเยอร์	หน้าที่	ควรมี	ห้ามมี	เปรียบเทียบ
Controller / Router	รับ HTTP request, ส่ง response	routing, mapping, validate input	business logic, SQL	แผนกต้อนรับโรงพยาบาล
Service	กฎทางธุรกิจ ประสานงาน	logic, การคำนวณ, เรียกหลาย repo	รายละเอียด HTTP หรือ DB	หมอ ตรวจวินิจฉัย
Repository	อ่าน/เขียนข้อมูล (CRUD)	query, persistence	business logic, HTTP	ห้องเวชระเบียน

src/routes/subject-route.js

```
import express from 'express';
import * as service from '../services/subject-service.js';

const router = express.Router();

router.get('/', async (req, res) => {
  res.json(await service.getAllSubjects());
});

router.get('/:id', async (req, res) => {
  const subject = await service.getSubjectById(req.params.id);
  if (!subject) return res.status(404).json({ message: 'Subject not found' });
  res.json(subject);
});

export default router;

// app.js:  app.use('/api/subjects', router)  → GET /api/subjects/5 จะเข้า router.get('/:id')
```

สัปดาห์ 5 · 3 ก.ย.

เชื่อมต่อ MySQL ด้วย mysql2

สร้าง connection pool, ทำ CRUD ใน repository และใช้ prepared statement กัน SQL Injection

1. ติดตั้ง driver

`npm install mysql2`

2. สร้าง Pool

pool คือที่จัดการ connection หลายเส้น ใช้ซ้ำได้ ไม่ต้องเปิดใหม่ทุก request

3. ส่ง SQL

`await pool.query(sql, params`
ได้ผลเป็น array
`[rows, fields]`

src/configs/db-config.js

```
import * as mysql from 'mysql2/promise';

const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'mysql@sit',
  database: 'sampledb',
});

export default pool;
```

ผลลัพธ์จาก pool.query

คำสั่ง	ได้อะไรใน `[result]`	ใช้ตรวจอะไร
`SELECT`	array ของแถว (`rows`)	`rows.length === 0` → ไม่พบ (404)
`INSERT`	ResultSetHeader	`result.insertId` → id ที่เพิ่งสร้าง
`UPDATE` / `DELETE`	ResultSetHeader	`result.affectedRows > 0` → สำเร็จ

src/repositories/subject-repo.js

```
import pool from '../configs/db-config.js';

export async function findAll() {
  const [rows] = await pool.query('SELECT * FROM subjects');
  return rows;
}

export async function findById(id) {
  const [rows] = await pool.query('SELECT * FROM subjects WHERE id = ?', [id]);
  return rows.length === 0 ? null : rows[0];
```

```
}

export async function create(subject) {
  const [result] = await pool.query(
    'INSERT INTO subjects (subject_code, subject_title, credit) VALUES (?, ?, ?)',
    [subject.code, subject.title, subject.credit]);
  return findById(result.insertId);
}

export async function update(id, subject) {
  const [result] = await pool.query(
    'UPDATE subjects SET subject_code = ?, subject_title = ?, credit = ? WHERE id = ?',
    [subject.code, subject.title, subject.credit, id]);
  if (result.affectedRows === 0) return null;
  return findById(id);
}

export async function remove(id) {
  const [result] = await pool.query('DELETE FROM subjects WHERE id = ?', [id]);
  return result.affectedRows > 0;
}
```

SQL Injection กับ Prepared Statement

ลองเป็นแฮกเกอร์

INTERACTIVE

ค่า id ที่ผู้ใช้ส่งมา (เช่น `/api/subjects/:id`)

```
1 OR 1=1
```

อันตราย ต่อสตริงเอง

```
'SELECT * FROM subjects WHERE id = ' +
id

→ SELECT * FROM subjects WHERE id = 1 OR
1=1
```

เงื่อนไข OR 1=1 เป็นจริงทุกแถว → ได้ข้อมูลทั้งตาราง

ปลอดภัย ใช้ `?` placeholder

```
pool.query('SELECT * FROM subjects WHERE
id = ?', [id])

→ SELECT * FROM subjects WHERE id = '1
OR 1=1'
```

ค่าถูก escape เป็นข้อมูล ไม่ใช่คำสั่ง SQL จึงหาแถวที่ id เท่ากับข้อความนั้นตรง ๆ

> **จำ:** ห้ามเอาค่าจากผู้ใช้ไปต่อสตริง SQL เด็ดขาด ให้ใส่ `?` แล้วส่งค่าเป็น array ตัวที่ 2 ของ `query()` เสมอ

ก่อนมี Express: อ่าน body จาก stream เอง

router.js · getBody()

```
function getBody(request) {
  return new Promise((resolve, reject) => {
    let body = '';
    request.on('data', chunk => { body += chunk.toString(); }); // รับทีละ chunk
    request.on('end', () => resolve(body ? JSON.parse(body) : null)); // ครบแล้ว parse
    request.on('error', err => reject(err));
  });
}
```

นี่คือสิ่งที่ `express.json()` ทำให้อัตโนมัติ เข้าใจไว้จะตอบข้อสอบทฤษฎีเรื่อง stream/body ได้

สัปดาห์ 8 · 1 ต.ค.

Validation และ Error Handling

ถ้า error ถูก throw แล้วไม่มีใคร catch, Node จะจบ process (`process.exit(1)`) ทั้ง server ล่ม การจัดการ error จึงเป็นเรื่องบังคับ

ทำไมต้องจัดการ error

- กัน app crash / downtime
- ข้อความ error ที่ผู้ใช้เข้าใจ
- ไม่หลุด stack trace / ข้อมูลลับ
- ช่วย debug, monitor, แจ้งเตือน

ประเภท error

- **Programming error** บั๊ก logic เช่น TypeError
- **Operational error** network timeout, DB ล่ม, ไม่พบไฟล์
- **Expected vs Unexpected** อันที่คาดไว้ต้องจัดการให้เรียบร้อย

Error propagation

error ที่เกิดในฟังก์ชันจะถูกส่ง**ขึ้นไปตาม call stack** (b → a → main) จนกว่าจะมีคน catch ถ้าไม่มีใคร catch เลยจนถึง global scope โปรแกรมจะหยุด ในระบบเลเยอร์ก็เหมือนกัน: Repository throw → Service ไม่ catch → Controller ส่ง `next(err)` → error middleware ตอบกลับ

Property ของ Error object

- `message` ข้อความอธิบาย
- `name` ชนิด เช่น "TypeError"
- `stack` stack trace (ไม่เป็นมาตรฐานแต่ทุกที่รองรับ)
- `code` รหัส system error เช่น "EACCES", "ENOENT"
- `errno` ตัวเลขของ system error

Error middleware ต่างจาก middleware ปกติ

ปกติ: `(req, res, next)` 3 ตัว

Error: `(err, req, res, next)` **4 ตัว** Express ดูจากจำนวน parameter

ต้องวางไว้**ท้ายสุด** หลัง route ทั้งหมด ถ้าไม่เขียนเลย Express จะตอบเป็น HTML พร้อม stack trace

src/middlewares/error-handler.js

```
export class AppError extends Error {
  static errors = { 400: 'BAD_REQUEST', 401: 'UNAUTHORIZED', 403: 'FORBIDDEN',
                    404: 'NOT_FOUND', 409: 'CONFLICT', 500: 'INTERNAL_SERVER_ERROR' };
  constructor(message, statusCode = 500) {
    super(message);
    this.statusCode = statusCode;
    this.error = AppError.errors[statusCode];
    this.isOperational = true;
    Error.captureStackTrace(this, this.constructor);
  }
}

// ไม่มี route ไหนตรง → 404
export function notFound(req, res, next) {
```

```
  next(new AppError(`Route ${req.method} ${req.originalUrl} not found`, 404));
}

// error-handling middleware (4 parameters) วางท้ายสุดใน app.js
export function errorHandler(err, req, res, next) {
  const status = err.statusCode || err.status || 500;
  res.status(status).json({
    error: err.error || AppError.errors[status] || err.code,
    statusCode: status,
    message: err.message,
    path: req.originalUrl,
    timestamp: new Date().toISOString(),
  });
}
```

> **async handler กับ error:** ใน Express 5 ถ้าฟังก์ชัน async throw หรือ Promise reject จะถูกส่งเข้า `next(err)` ให้อัตโนมัติ แต่ใน Express 4 ต้องห่อ `try { … } catch (e) { next(e) }` เอง ไม่งั้น request ค้างและอาจ crash ดูเวอร์ชันใน package.json ของ template ก่อน ถ้าไม่แน่ใจให้ใส่ try/catch ไว้ทุก route ปลอดภัยสุด

Status code ที่ต้องจำ

Code	ชื่อ	ใช้เมื่อ
200	OK	สำเร็จ (GET, PUT, DELETE)
201	Created	สร้าง resource ใหม่สำเร็จ (POST)
400	Bad Request	client ส่งข้อมูลผิด เช่น ไม่มี body หรือขาด field ที่จำเป็น
401	Unauthorized	ยืนยันตัวตนไม่ผ่าน (ไม่ได้ login / token ผิด)
403	Forbidden	ยืนยันตัวตนแล้ว แต่**ไม่มีสิทธิ์**เข้าถึง
404	Not Found	ไม่มี resource นั้น
409	Conflict	ขัดกับสถานะปัจจุบัน เช่น ข้อมูลซ้ำ (unique)
412	Precondition Failed	เงื่อนไขใน request header เป็นเท็จ
500	Internal Server Error	error ทั่วไปฝั่ง server
503	Service Unavailable	บริการใช้งานไม่ได้ชั่วคราว

สัปดาห์ 9–10 · 8 ต.ค.

Pagination และ DTO

แบ่งข้อมูลเป็นหน้าเพื่อลดภาระ DB, หน่วยความจำ และ network และใช้ DTO คุมหน้าตาข้อมูลที่ส่งออกไป

สูตรหัวใจ

> **OFFSET = (page - 1) × limit**

page	limit	offset	ได้แถวที่
1	10	0	1–10
2	10	10	11–20
3	10	20	21–30
9	10	80	81–90

เครื่องคิด Pagination

INTERACTIVE

page: 2

limit: 20

totalItems: 138

ลองใส่ page = -3 หรือ limit = 500 เพื่อดูว่า Math.max / Math.min ช่วยกันค่าผิดยังไง

```
page  = Math.max(parseInt("2"), 1)   → 2
limit = Math.min(parseInt("20"), 100) → 20
offset = (2 - 1) × 20 = 20

SELECT * FROM products LIMIT 20 OFFSET 20
→ ได้แถวที่ 21–40 (20 แถว)

{
  "pagination": {
    "total": 138,
    "page": 2,
    "limit": 20,
    "totalPages": 7,
    "hasNextPage": true,
    "hasPrevPage": true
  }
}
```

ลำดับการทำงาน

1. client ส่ง `GET /api/products?page=2&limit=20`
2. middleware ตรวจ page/limit แล้วคำนวณ offset เก็บไว้ใน `req.pagination`
3. repository query ด้วย `LIMIT ? OFFSET ?` และ `COUNT(*)` หาจำนวนทั้งหมด
4. ส่ง data พร้อม pagination metadata กลับไป

Best practices

- page ต้อง ≥ 1 → `Math.max(parseInt(page), 1)`
- จำกัด limit สูงสุด เช่น 100 → `Math.min(parseInt(limit), 100)`
- ใช้ prepared statement
- ทำ index คอลัมน์ที่ใช้ filter/sort
- ถ้าผู้ใช้เข้าหน้าลึกมาก ๆ พิจารณา **cursor pagination**

Metadata ที่ควรส่ง

- `totalPages = Math.ceil(total / limit)`
- `hasNextPage = page * limit < total`
- `hasPrevPage = page > 1`

ต้องรู้ totalItems ก่อน จึงต้อง query `SELECT COUNT(*) AS total` แยกอีกครั้ง

src/middlewares/pagination-middleware.js

```
export const queryParser = () => (req, res, next) => {
  const query = req.query || {};
  if (query.page === undefined || query.limit === undefined) {
    req.pagination = null;            // ไม่ขอแบ่งหน้า → ส่งทั้งหมด
    return next();
  }
  const page  = Math.max(parseInt(query.page)  || 1, 1);     // ไม่ให้ติดลบ / NaN
  const limit = Math.min(parseInt(query.limit) || 20, 100); // ไม่เกิน 100
  const offset = (page - 1) * limit;    // ใช้ limit ที่ผ่านการตรวจแล้ว
  req.pagination = { page, limit, offset };
  next();
};

// app.js: app.use(queryParser())  ← ต้องวางก่อน router
```

DTO (Data Transfer Object)

DTO คือ object ที่กำหนด**รูปร่างของข้อมูลที่ส่งข้ามเลเยอร์/ออกไปทาง API** แยก "สัญญาของ API" ออกจาก "โครงสร้างตารางในฐานข้อมูล" ข้อมูลจะถูก map จาก entity → DTO ผ่าน **mapper** และจาก entity เดียวทำ DTO ได้หลายแบบ (เช่น summary กับ detail)

Entity (ในฐานข้อมูล)

```
id, name, email,
password_hash, role,
created_at
```

Response DTO (ส่งออก)

```
id, name, email, role
```

ห้ามส่ง `password_hash` ออกไปเด็ดขาด

ทำไมใช้ DTO

- กันไม่ให้ field ลับหลุดออกไปโดยไม่ตั้งใจ
- response หน้าตาสม่ำเสมอ
- validate และเขียนเอกสาร API ง่ายขึ้น
- ลด coupling ระหว่าง DB กับ client (เปลี่ยนชื่อคอลัมน์ได้โดย API ไม่เปลี่ยน)

src/mappers/product-mapper.js

```
export default class ProductMapper {
  // ข้อมูลย่อ (ใช้ตอน list)
  static toSummaryDTO(entity) {
    return {
      id: entity.productCode,
      name: entity.productName,
      scale: entity.productScale,
      retailPrice: entity.MSRP,
    };
  }
  // ข้อมูลเต็ม (ใช้ตอนดูทีละชิ้น): เปลี่ยนชื่อ field แล้วตัดของเดิมออก
  static toDetailDTO(entity) {
    const { MSRP, buyPrice, ...data } = entity;
    return { ...data, retailPrice: MSRP, price: buyPrice };
  }
  static map(entity, view = 'full') {
    return view === 'summary' ? this.toSummaryDTO(entity) : this.toDetailDTO(entity);
  }
  static maps(entities, view = 'summary') {
    const list = entities.map(e => this.map(e, view));
    if (entities.totalItems !== undefined) list.totalItems = entities.totalItems; // พก total ไปด้วย
    return list;
  }
}
```

src/mappers/api-response.js

```
export function apiResponse(res, data, pagination = null, status = 200) {
  if (!pagination) {
    return res.status(status).json({ success: true, data, timestamp: new Date().toISOString() });
  }
  const { page, limit } = pagination;
  const total = data.totalItems;
  return res.status(status).json({
```

```
    success: true,
    data,
    pagination: {
      total, page, limit,
      totalPages: Math.ceil(total / limit),
      hasNextPage: page * limit < total,
      hasPrevPage: page > 1,
    },
    timestamp: new Date().toISOString(),
  });
}
```

> **รูปแบบ response ในข้อสอบ** ให้ยึดตาม spec ของตัวเองทุกตัวอักษร (ชื่อ key เช่น `hasPrevPage` หรือ `hasPreviousPage`, `total` หรือ `totalItems`) เพราะ test runner ตรวจตาม format เป๊ะ สไลด์สองหน้ายังใช้ชื่อไม่เหมือนกันเลย

เสริม: Filtering และ Sorting

หัวข้อนี้อยู่ใน course outline แต่ไม่มีสไลด์ หลักการที่ใช้ต่อยอดจาก pagination คือรับค่าจาก query string แล้วสร้าง SQL อย่างปลอดภัย

ตัวอย่าง: GET /api/products?name=car&sort=MSRP&order=desc&page=1&limit=10

```
const SORTABLE = ['productName', 'MSRP', 'buyPrice'];   // whitelist คอลัมน์

export async function findAll({ name, sort, order } = {}, pagination) {
  let sql = 'SELECT * FROM products';
  const params = [];
  if (name) { sql += ' WHERE productName LIKE ?'; params.push(`%${name}%`); } // filter
  if (SORTABLE.includes(sort)) {                                              // sort
    sql += ` ORDER BY ${sort} ${order === 'desc' ? 'DESC' : 'ASC'}`;
  }
  if (pagination) { sql += ' LIMIT ? OFFSET ?'; params.push(pagination.limit, pagination.offset); }
  const [rows] = await pool.query(sql, params);
  return rows;
}
```

ชื่อคอลัมน์ใส่ `?` ไม่ได้ (? ใช้กับค่าเท่านั้น) จึงต้องเช็กกับ whitelist ก่อนเอาไปต่อสตริง ส่วนค่าที่ใช้ค้นหายังใช้ `?` ตามปกติ และ ORDER BY ต้องมาก่อน LIMIT

จากคาบซ้อมสอบ 15 ต.ค.

เตรียมสอบปฏิบัติ

ได้ zip template โปรเจกต์ (Express ESM) ของแต่ละคน + ไฟล์ SQL สร้างฐานข้อมูล + spec (.md) + โฟลเดอร์ test ตรวจคะแนนเองได้ทีละข้อ

ขั้นตอนวันสอบ

1. 15 นาทีแรกยังมีเน็ต: แตก zip ระวังโฟลเดอร์ซ้อน, เปิดใน WebStorm แล้ว `npm install`, โหลดเอกสาร/โค้ดเก่าของตัวเองเก็บในเครื่อง
2. หลัง 15 นาที**ปิดเน็ต** เปิดเน็ตอีก = ทุจริต
3. รัน SQL script สร้าง DB (WebStorm: คลิกขวา data source → Run SQL Script) แล้ว refresh ดูตาราง
4. ตั้ง student ID ใน `app.js` → route ทั้งหมดอยู่ใต้ `/api/<รหัสนักศึกษา>`
5. รันโปรเจกต์ที่พอร์ต 3000 แล้วทดสอบ: `node test-runner.mjs <รหัส>`

กติกาที่ต้องจำ

- ใช้เครื่องคณะ (Windows) คนใช้ Mac ควรซ้อมบน Windows
- **ห้ามติดตั้ง dependency เพิ่ม** ใช้เฉพาะที่ให้มา
- เพิ่มไฟล์/โฟลเดอร์เองได้ตามสะดวก
- template มี config, repo/service/route ตัวอย่าง, mapper, pagination middleware แต่ **error handling ต้องเขียนเอง**
- ไม่มี AI ช่วย ใช้ได้แค่เอกสารที่โหลดไว้
- `timestamp` อาจถูกตรวจว่าเป็นเวลาปัจจุบันจริง อย่า hard-code

เช็กลิสต์ก่อนส่ง

- [ ] `"type": "module"` และ import ทุกไฟล์มี `.js` ต่อท้าย
- [ ] ชื่อ DB / user / password ใน db-config ตรงกับเครื่อง
- [ ] ลำดับใน app.js: `express.json()` → `queryParser()` → routers → notFound → errorHandler
- [ ] path ตรง spec: `/api/<รหัส>/…` พหูพจน์ตามที่ spec เขียน
- [ ] status code ถูก: POST = 201, ไม่พบ = 404, ข้อมูลไม่ครบ = 400, ซ้ำ = 409
- [ ] response format ตรง spec ทุก key (`success` , `data` , `pagination` , `timestamp`)
- [ ] DTO ตัด/เปลี่ยนชื่อ field ตามที่ spec สั่ง
- [ ] รัน test runner ผ่าน แล้วรันซ้ำหลังแก้ทุกครั้ง

โครงโปรเจกต์ครบชุด (เก็บไว้ดูตอนสอบ)

ตัวอย่างใช้ตาราง `brands` แบบที่เจอในคาบซ้อมสอบ (id, name, country, website, …) เปลี่ยนชื่อตาราง/field ตาม spec ของตัวเอง

```
src/
├── app.js
├── configs/db-config.js
├── middlewares/
│   ├── error-handler.js        ← เขียนเอง (ดูสัปดาห์ Error Handling)
│   └── pagination-middleware.js
```

```
├── mappers/
│   ├── api-response.js
│   └── brand-mapper.js
├── repositories/brand-repo.js
├── services/brand-service.js
└── routes/brand-route.js
```

src/app.js

```
import express from 'express'
import { queryParser } from './middlewares/pagination-middleware.js'
import { notFound, errorHandler } from './middlewares/error-handler.js'
import brandRouter from './routes/brand-route.js'

const STUDENT_ID = '66130500001'          // ← รหัสของตัวเอง
const PORT = 3000
const app = express()

app.use(express.json())                   // 1) parse body
app.use(queryParser())                    // 2) pagination (ก่อน router)

const BASE = `/api/${STUDENT_ID}`
app.get(`${BASE}/student-info`, (req, res) => {
  res.json({ success: true, data: { studentId: STUDENT_ID }, timestamp: new Date().toISOString() })
})
app.use(`${BASE}/brands`, brandRouter)    // 3) routers

app.use(notFound)                         // 4) ไม่มี route ตรง
app.use(errorHandler)                     // 5) error handler ท้ายสุด

app.listen(PORT, () => console.log(`Running at http://localhost:${PORT}${BASE}`))
```

src/configs/db-config.js

```
import * as mysql from 'mysql2/promise'

const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'mysql@sit',
  database: 'mobile_distributor',      // ← ตามไฟล์ SQL ของ spec
})

export default pool
```

src/middlewares/error-handler.js

```
export class AppError extends Error {
  static errors = { 400: 'BAD_REQUEST', 401: 'UNAUTHORIZED', 403: 'FORBIDDEN',
                    404: 'NOT_FOUND', 409: 'CONFLICT', 500: 'INTERNAL_SERVER_ERROR' }
  constructor(message, statusCode = 500) {
    super(message)
    this.statusCode = statusCode
```

```
    this.error = AppError.errors[statusCode]
    this.isOperational = true
    Error.captureStackTrace(this, this.constructor)
  }
}

export function notFound(req, res, next) {
  next(new AppError(`Route ${req.method} ${req.originalUrl} not found`, 404))
}

export function errorHandler(err, req, res, next) {
  // แปลง error จาก MySQL ให้เป็น status ที่เหมาะสม
  if (err.code === 'ER_DUP_ENTRY') { err.statusCode = 409; err.error = 'CONFLICT' }
  if (err.type === 'entity.parse.failed') { err.statusCode = 400; err.message = 'Invalid JSON body' }
  const status = err.statusCode || err.status || 500
  res.status(status).json({
    success: false,
    error: err.error || AppError.errors[status] || 'ERROR',
    statusCode: status,
    message: err.message,
    path: req.originalUrl,
    timestamp: new Date().toISOString(),
  })
}
```

src/middlewares/pagination-middleware.js

```
export const queryParser = () => (req, res, next) => {
  const query = req.query || {}
  if (query.page === undefined || query.limit === undefined) {
    req.pagination = null
    return next()
  }
  const page = Math.max(parseInt(query.page) || 1, 1)
  const limit = Math.min(parseInt(query.limit) || 20, 100)
  const offset = (page - 1) * limit
  req.pagination = { page, limit, offset }
  next()
}
```

src/repositories/brand-repo.js

```
import db from '../configs/db-config.js'
import { AppError } from '../middlewares/error-handler.js'

const TABLE = 'brands'
const ENTITY = 'Brand'

export async function count() {
  const [[{ total }]] = await db.query(`SELECT COUNT(*) AS total FROM ${TABLE}`)
  return total
}

export async function findAll(pagination) {
  if (!pagination) {
```

```
    const [rows] = await db.query(`SELECT * FROM ${TABLE}`)
    return rows
  }
  const [rows] = await db.query(`SELECT * FROM ${TABLE} LIMIT ? OFFSET ?`,
    [pagination.limit, pagination.offset])
  rows.totalItems = await count()
  return rows
}

export async function findById(id) {
  const [rows] = await db.query(`SELECT * FROM ${TABLE} WHERE id = ?`, [id])
  if (rows.length === 0) throw new AppError(`${ENTITY} with id '${id}' was not found`, 404)
  return rows[0]
}

export async function create(data) {
  const [result] = await db.query(`INSERT INTO ${TABLE} SET ?`, [data])
  return findById(result.insertId)
}

export async function update(id, data) {
  await findById(id)                                  // ไม่มี → 404
  await db.query(`UPDATE ${TABLE} SET ? WHERE id = ?`, [data, id])
  return findById(id)
}

export async function remove(id) {
  const existing = await findById(id)                 // ไม่มี → 404
  await db.query(`DELETE FROM ${TABLE} WHERE id = ?`, [id])
  return existing
}
```

src/mappers/brand-mapper.js

```
export default class BrandMapper {
  static toSummaryDTO(e) {
    return { id: e.id, name: e.name, countryOfOrigin: e.country }
  }
  static toDetailDTO(e) {
    const { country, created_at, ...rest } = e      // ตัด/เปลี่ยนชื่อ field ตาม spec
    return { ...rest, countryOfOrigin: country }
  }
  static map(e, view = 'full') {
    return view === 'summary' ? this.toSummaryDTO(e) : this.toDetailDTO(e)
  }
  static maps(list, view = 'summary') {
    const out = list.map(e => this.map(e, view))
    if (list.totalItems !== undefined) out.totalItems = list.totalItems
    return out
  }
}
```

src/mappers/api-response.js

```
export function apiResponse(res, data, pagination = null, status = 200) {
  if (!pagination) {
    return res.status(status).json({ success: true, data, timestamp: new Date().toISOString() })
  }
  const { page, limit } = pagination
  const total = data.totalItems
  return res.status(status).json({
    success: true,
    data,
    pagination: {
      total, page, limit,
      totalPages: Math.ceil(total / limit),
      hasNextPage: page * limit < total,
      hasPrevPage: page > 1,
    },
    timestamp: new Date().toISOString(),
  })
}
```

src/services/brand-service.js

```
import * as repo from '../repositories/brand-repo.js'
import mapper from '../mappers/brand-mapper.js'
import { AppError } from '../middlewares/error-handler.js'

export async function getAll(pagination = null, view = 'summary') {
  return mapper.maps(await repo.findAll(pagination), view)
}

export async function getById(id, view = 'full') {
  return mapper.map(await repo.findById(id), view)
}

function validate(body) {
  if (!body || Object.keys(body).length === 0) throw new AppError('Request body is empty', 400)
  if (!body.name) throw new AppError('name is required', 400)
}

export async function create(body) {
  validate(body)
  const { name, country, website } = body               // รับเฉพาะ field ที่อนุญาต
  return mapper.map(await repo.create({ name, country, website }))
}

export async function update(id, body) {
  validate(body)
  const { name, country, website } = body
  return mapper.map(await repo.update(id, { name, country, website }))
}

export async function remove(id) {
  return mapper.map(await repo.remove(id))
}
```

src/routes/brand-route.js

```
import express from 'express'
import * as service from '../services/brand-service.js'
import { apiResponse } from '../mappers/api-response.js'

const router = express.Router()

// try/catch + next(e) ใช้ได้ทั้ง Express 4 และ 5
router.get('/', async (req, res, next) => {
  try {
    const view = req.query.view || 'summary'
    const data = await service.getAll(req.pagination, view)
    apiResponse(res, data, req.pagination)
  } catch (e) { next(e) }
})

router.get('/:id', async (req, res, next) => {
  try {
    apiResponse(res, await service.getById(req.params.id, req.query.view || 'full'))
  } catch (e) { next(e) }
})

router.post('/', async (req, res, next) => {
  try {
    apiResponse(res, await service.create(req.body), null, 201)
  } catch (e) { next(e) }
})

router.put('/:id', async (req, res, next) => {
  try {
    apiResponse(res, await service.update(req.params.id, req.body))
  } catch (e) { next(e) }
})

router.delete('/:id', async (req, res, next) => {
  try {
    apiResponse(res, await service.remove(req.params.id))
  } catch (e) { next(e) }
})

export default router
```

ซ้อมข้อสอบทฤษฎี

แบบฝึกหัด 5 ตัวเลือก

รูปแบบเดียวกับข้อสอบจริง (ตอบถูกข้อเดียว) ทำให้ครบก่อน แล้วค่อยเปิดเฉลยท้ายเล่ม

1. ใน middleware ถ้าไม่เรียก next() และไม่ส่ง response จะเกิดอะไร

ก. server crash ทันที
ข. Express ส่ง 200 ให้อัตโนมัติ
ค. request ค้าง client รอจน timeout
ง. ไปที่ error handler
จ. ไปที่ middleware ถัดไปเอง

2. ข้อใดถูกต้องเกี่ยวกับ ESM เมื่อเทียบกับ CommonJS

ก. รองรับ tree-shaking ให้ bundler ตัดโค้ดที่ไม่ใช้ออก
ข. ต้องใช้ module.exports
ค. ใช้ require() ในการ import
ง. โหลดแบบ synchronous เท่านั้น
จ. ใช้ได้เฉพาะใน Node.js

3. ควรวาง error-handling middleware ไว้ที่ใดใน app.js

ก. ท้ายสุด หลัง route และ middleware อื่นทั้งหมด
ข. ที่ใดก็ได้
ค. บนสุดก่อน express.json()
ง. ใน repository
จ. ก่อน router

4. ผู้ใช้ login แล้ว แต่ไม่มีสิทธิ์เข้าถึง resource ควรตอบ status ใด

ก. 409 Conflict
ข. 404 Not Found
ค. 400 Bad Request
ง. 403 Forbidden
จ. 401 Unauthorized

5. request GET /api/subjects/5?page=2 กับ route /api/subjects/:id ข้อใดถูก

ก. req.params.id = '5' และ req.query.page = '2'
ข. req.params.id = 5 (number)
ค. req.body.page = '2'
ง. req.query.id = '5'
จ. req.params.page = '2'

6. สูตรใดใช้หา hasNextPage ได้ถูกต้อง

ก. total % limit === 0
ข. page > 1
ค. page < limit
ง. offset > total
จ. page * limit < total

7. ข้อใดอธิบายหลัก Stateless ของ REST ได้ถูกต้อง

ก. ต้องใช้ HTTP method แค่ GET
ข. server ต้องจำผู้ใช้จาก request ก่อนหน้า
ค. แต่ละ request ต้องมีข้อมูลครบในตัวเอง server ไม่เก็บ session state ของ client
ง. client ห้ามเก็บข้อมูลใด ๆ
จ. response ทุกอันต้อง cache ได้

8. totalItems = 138, limit = 20 จะมีทั้งหมดกี่หน้า

ก. 138
ข. 7
ค. 6.9
ง. 6
จ. 8

9. express.json() คืออะไร

ก. built-in middleware ที่ parse JSON body ไปไว้ใน req.body
ข. error-handling middleware
ค. router สำหรับ path /json
ง. third-party library ต้องติดตั้งเพิ่ม
จ. ฟังก์ชันส่ง response เป็น JSON

10. SOAP Web Service ใช้อะไรอธิบายรูปแบบ request/response

ก. WSDL ซึ่งเป็น XML
ข. GraphQL schema
ค. OpenAPI YAML
ง. JSON Schema
จ. HTML form

11. ข้อใดเป็น Operational Error

ก. TypeError จากการเรียก undefined.name
ข. ลืม return ค่าในฟังก์ชัน
ค. loop ไม่มีวันจบ
ง. ตั้งชื่อตัวแปรผิด
จ. ฐานข้อมูลเชื่อมต่อไม่ได้

12. ภาษาใดใช้ตัวแปลแบบ Hybrid (compile เป็น byte code แล้วรันบน VM)

ก. Assembly
ข. C++
ค. C
ง. Java
จ. Go

13. page = 4, limit = 15 ค่า OFFSET คือเท่าใด

ก. 60
ข. 45
ค. 56
ง. 15
จ. 4

14. หลักการข้อใดของ REST เป็น "optional"

ก. Layered System
ข. Stateless
ค. Code on Demand
ง. Uniform Interface
จ. Client–Server

15. เมื่อเรียก next(err) Express จะทำอย่างไร

ก. รีสตาร์ต server
ข. ส่ง 200 กลับทันที
ค. ข้าม middleware/route ปกติทั้งหมด ไปหา error-handling middleware
ง. ไป middleware ถัดไปตามปกติ
จ. throw error ซ้ำ

16. Layered System มีประโยชน์ข้อใด "น้อยที่สุด"

ก. Separation of Concerns
ข. ทำให้ server จำ session ของผู้ใช้ได้
ค. เพิ่มความปลอดภัย
ง. แต่ละชั้นแก้ไขได้อิสระ
จ. เพิ่ม cache / load balancer ได้โดย client ไม่ต้องแก้

17. ข้อใดกล่าวถูกต้องเกี่ยวกับ Entity และ DTO

ก. Entity กับ DTO ต้องมี field เหมือนกันเสมอ
ข. DTO เก็บอยู่ในฐานข้อมูล
ค. entity หนึ่งมี DTO ได้แบบเดียว
ง. DTO ใช้เฉพาะ request ไม่ใช้กับ response
จ. Entity แทนข้อมูลที่เก็บใน DB ส่วน DTO แทนข้อมูลที่ API รับ/ส่ง

18. ข้อใด "ไม่ใช่" หนึ่งใน 4 เสาหลักของ Uniform Interface

ก. Self-Descriptive Messages
ข. Manipulation via Representations
ค. Stateless session
ง. Resource Identification
จ. HATEOAS

19. URI ใดออกแบบถูกหลัก REST ที่สุด สำหรับดึง orders ของสินค้า id 1

ก. GET /api/products/1/orders
ข. GET /api/orders/listForProduct/1
ค. POST /api/products/orders/get
ง. GET /api/getOrdersByProduct?id=1
จ. GET /api/product/1/order

20. ใน ES Modules ใช้อะไรแทน __dirname และ __filename

ก. global.__path
ข. process.cwd()
ค. require.resolve
ง. module.exports
จ. import.meta.url

21. ข้อใดถูกต้องเกี่ยวกับ res.status(201)

ก. ใช้ได้กับ GET เท่านั้น
ข. ตั้ง status code อย่างเดียว ยังไม่ส่ง ต้องต่อด้วย .json() หรือ .send()
ค. ส่ง body เป็น 201
ง. ทำให้ Express ข้ามไป error handler
จ. ส่ง response ทันที

22. ใน MPA (Multi-Page Application) เมื่อผู้ใช้เปลี่ยนหน้าจะเกิดอะไร

ก. ขอ JSON แล้วอัปเดตบางส่วน
ข. โหลดเฉพาะ CSS
ค. โหลดหน้าใหม่จาก server ทั้งหน้า (full page refresh)
ง. ใช้ WebSocket เสมอ
จ. ไม่ติดต่อ server เลย

23. วิธีใดป้องกัน SQL Injection ได้ถูกต้อง

ก. pool.query(`SELECT * FROM t WHERE id = ${id}`)
ข. pool.query('SELECT * FROM t WHERE id = ' + id)
ค. ตรวจว่า id ไม่มีช่องว่างแล้วต่อสตริง
ง. ใช้ GET แทน POST
จ. pool.query('SELECT * FROM t WHERE id = ?', [id])

24. ชั้นใดไม่ควรมี business logic และ SQL

ก. Repository
ข. Service และ Repository
ค. Service
ง. Controller
จ. ทุกชั้นมีได้

25. REST ย่อมาจากอะไร

ก. Representational Server Technology
ข. Request–Response State Transaction
ค. Representational State Transfer
ง. Remote Execution State Transfer
จ. Resource Endpoint Service Transport

26. Math.min(parseInt(limit), 100) ใน pagination middleware มีไว้ทำอะไร

ก. จำกัดไม่ให้ขอเกิน 100 แถวต่อหน้า
ข. คำนวณ offset
ค. ทำให้ page ไม่ติดลบ
ง. นับจำนวนแถวทั้งหมด
จ. แปลง limit เป็น string

27. URL ?fav_subj=A&fav_subj=B ต้องใช้คำสั่งใดจึงได้ทั้งสองค่า

ก. searchParams.get('fav_subj')
ข. searchParams.has('fav_subj')
ค. url.pathname
ง. request.params.fav_subj
จ. searchParams.getAll('fav_subj')

28. สร้างข้อมูลใหม่แต่ข้อมูลซ้ำกับคอลัมน์ UNIQUE ที่มีอยู่ ควรตอบ status ใด

ก. 400 Bad Request
ข. 500 Internal Server Error
ค. 404 Not Found
ง. 403 Forbidden
จ. 409 Conflict

29. error-handling middleware ของ Express มี signature แบบใด

ก. (err, req, res, next)
ข. (err, res)
ค. (req, res, next)
ง. (req, res)
จ. (next, err, req, res)

30. ผลของ '/users/123'.split('/').filter(Boolean) คือข้อใด

ก. [true, true]
ข. ['', 'users', '123']
ค. ['users/123']
ง. ['/users', '/123']
จ. ['users', '123']

31. จุดประสงค์หลักของ DTO คือข้อใด

ก. สร้างตารางในฐานข้อมูล
ข. เพิ่มความเร็วการ query
ค. แทนที่ repository
ง. เข้ารหัสรหัสผ่าน
จ. ควบคุมหน้าตาข้อมูลที่ส่งออก ไม่ให้ field ลับอย่าง password_hash หลุดไป

32. หลัง DELETE ควรตรวจค่าใดเพื่อรู้ว่าลบสำเร็จ

ก. result.changedId
ข. fields.length
ค. result.insertId
ง. rows.length
จ. result.affectedRows > 0

33. ข้อใดถูกต้องเกี่ยวกับ Node.js

ก. รันได้เฉพาะบน Linux
ข. เป็น framework บน Express
ค. เป็นภาษาโปรแกรมใหม่
ง. เป็น compiler แปล JS เป็น .exe
จ. เป็น runtime สำหรับรัน JavaScript ฝั่ง server

34. ถ้า exception ถูก throw แล้วไม่มีใคร catch ใน Node.js จะเกิดอะไร

ก. process จบการทำงาน (process.exit(1)) server ล่ม
ข. Event loop รีสตาร์ตเอง
ค. Node ข้ามไปทำบรรทัดถัดไป
ง. ได้ status 500 อัตโนมัติ
จ. error ถูกเก็บลง log แล้วทำงานต่อ

35. ใน http.ServerResponse เมธอดใดบอกว่าส่ง header และ body ครบแล้ว

ก. response.writeHead()
ข. response.write()
ค. response.setHeader()
ง. response.listen()
จ. response.end()

36. await pool.query(...) ของ mysql2/promise คืนค่าอะไร

ก. Promise ที่ต้อง await อีกรอบ
ข. object { rows }
ค. string JSON
ง. จำนวนแถว
จ. array [rows, fields]

37. ข้อดีของ Interpreter เมื่อเทียบกับ Compiler คือข้อใด

ก. รันเร็วกว่า
ข. รันข้ามแพลตฟอร์มได้ดี ใครมี interpreter ก็รันได้
ค. ซ่อน source code ได้
ง. ตรวจ error ทั้งโปรแกรมก่อนรัน
จ. ไม่ต้องมีโปรแกรมแปลในเครื่อง

38. ชั้นใดเปรียบเหมือน "หมอ" ที่วินิจฉัยและตัดสินใจ

ก. Controller
ข. Service
ค. Middleware
ง. Repository
จ. Database

เฉลยแบบฝึกหัด

ข้อ	ตอบ	เหตุผล
1	**ค**	ต้องเลือกอย่างใดอย่างหนึ่ง: เรียก next() ส่งต่อ หรือส่ง response จบ
2	**ก**	ESM ใช้ import/export, parse แบบ static, ใช้ได้ทั้งเบราว์เซอร์และ Node และรองรับ tree-shaking
3	**ก**	ต้องอยู่ท้ายสุด จึงรับ error จากทุก route ได้
4	**ง**	401 = ยืนยันตัวตนไม่ผ่าน, 403 = ผ่านแล้วแต่ไม่มีสิทธิ์
5	**ก**	path variable อยู่ใน req.params, query string อยู่ใน req.query และเป็น string เสมอ
6	**จ**	ถ้าจำนวนที่แสดงไปแล้ว (page × limit) ยังน้อยกว่าทั้งหมด แปลว่ายังมีหน้าถัดไป
7	**ค**	Stateless: client ต้องเก็บ session state เอง และส่งข้อมูลที่จำเป็นมาในทุก request
8	**ข**	Math.ceil(138 / 20) = Math.ceil(6.9) = 7
9	**ก**	เป็น built-in middleware ของ Express
10	**ก**	SOAP ใช้ XML และอธิบายด้วย Web Service Definition Language (WSDL)
11	**จ**	operational = ปัญหาจากสภาพแวดล้อม (network, DB, file) ไม่ใช่บั๊กในโค้ด
12	**ง**	Java, Kotlin, C# compile เป็น byte code แล้วให้ VM (interpreter + JIT) รัน
13	**ข**	(4 – 1) × 15 = 45
14	**ค**	Code on Demand (server ส่ง script ให้ client รัน) เป็นข้อเดียวที่ไม่บังคับ
15	**ค**	next(err) คือสัญญาณให้กระโดดไป error handler
16	**ข**	การจำ session ขัดกับ Stateless และไม่ใช่ประโยชน์ของ Layered System
17	**จ**	entity เดียว map เป็น DTO ได้หลายแบบ เช่น summary กับ detail
18	**ค**	Stateless เป็น constraint แยกต่างหาก ไม่ใช่เสาหลักของ Uniform Interface
19	**ก**	ใช้คำนามพหูพจน์ ไม่มีกริยาใน URI และเป็น sub-collection ของ products/{id}
20	**จ**	ESM ไม่มี __dirname/__filename ให้ใช้ import.meta.url

21	**ข**	res.status() คืน res กลับมาให้ chain ต่อ เช่น res.status(201).json(obj)
22	**ค**	SPA ต่างหากที่โหลดครั้งเดียวแล้วขอ JSON ผ่าน AJAX
23	**จ**	prepared statement (?) ทำให้ค่าเป็นข้อมูล ไม่ถูกตีความเป็นคำสั่ง SQL
24	**ง**	Controller รับ/ส่ง HTTP แล้ว delegate ให้ service เท่านั้น
25	**ค**	REST = REpresentational State Transfer คิดโดย Roy Fielding
26	**ก**	Math.max(page, 1) กันหน้าติดลบ, Math.min(limit, 100) กันขอเยอะเกิน
27	**จ**	get() ได้ค่าแรกค่าเดียว getAll() ได้ array ทุกค่า
28	**จ**	409 = request ขัดกับสถานะปัจจุบันของ resource
29	**ก**	Express รู้ว่าเป็น error handler จากการมี 4 parameter
30	**จ**	split ได้ ['', 'users', '123'] แล้ว filter(Boolean) ตัดสตริงว่างทิ้ง
31	**จ**	DTO แยกสัญญาของ API ออกจาก entity ในฐานข้อมูล
32	**จ**	affectedRows บอกจำนวนแถวที่ได้รับผลกระทบ; insertId ใช้หลัง INSERT
33	**จ**	Node.js ไม่ใช่ภาษา เป็น runtime environment ที่รันได้หลาย OS
34	**ก**	นี่คือเหตุผลที่ต้องมี error handling
35	**จ**	end() ปิด response; write() ส่งทีละ chunk; writeHead() ส่ง status + header
36	**จ**	จึงนิยมเขียน const [rows] = await pool.query(...)
37	**ข**	Interpreter portable สูงและ debug ไว แต่ช้ากว่าและต้องแจก source code
38	**ข**	Controller = แผนกต้อนรับ, Service = หมอ, Repository = ห้องเวชระเบียน

สรุปจากสไลด์ INT161 สัปดาห์ 1–7 และบันทึกคาบเรียนกลุ่ม B ระหว่าง 6 ส.ค.–15 ต.ค. 2569 · รายละเอียดการสอบให้ยึดประกาศในช่อง General ของรายวิชาเป็นหลัก